机器学习实验课5

SVM支持向量机

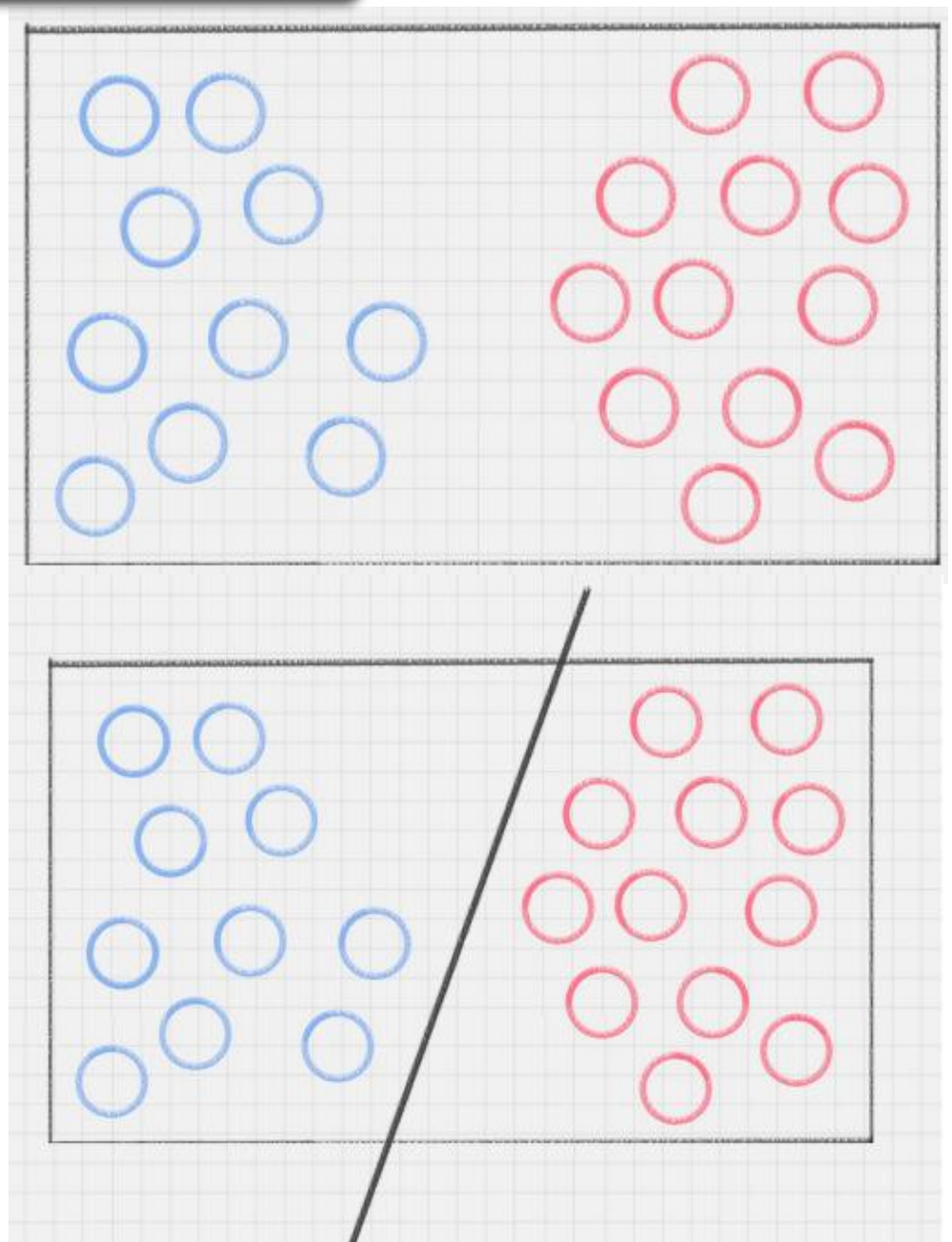


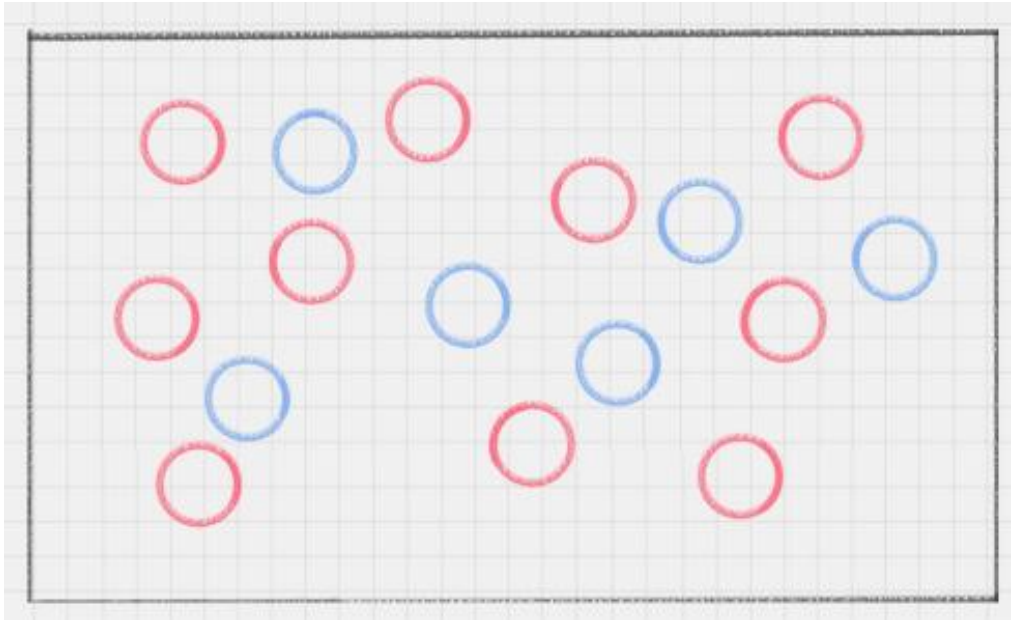
致谢：梁威鹏

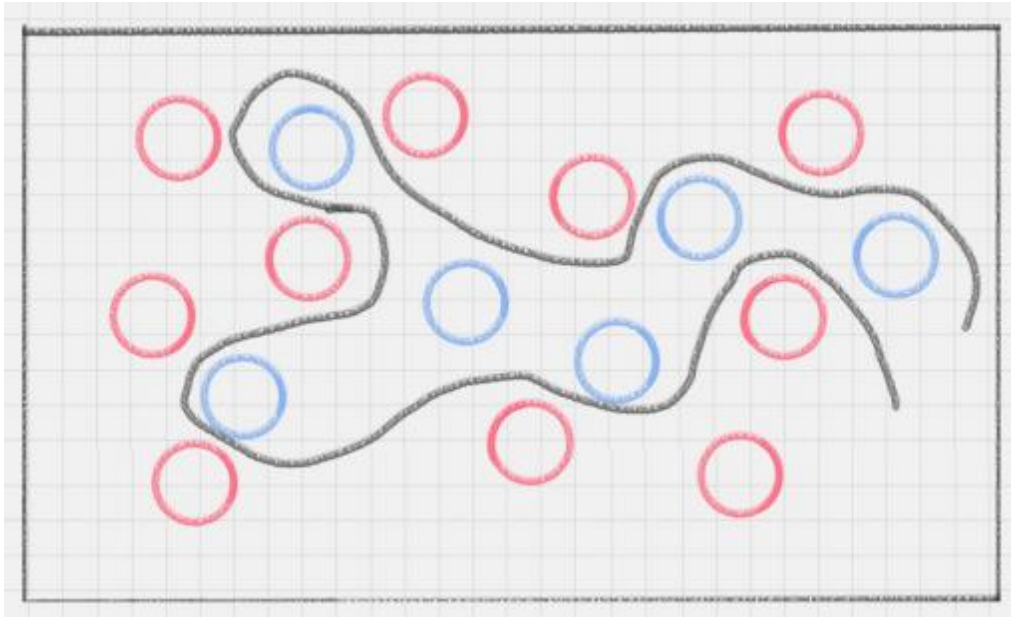


SVM支持向量机

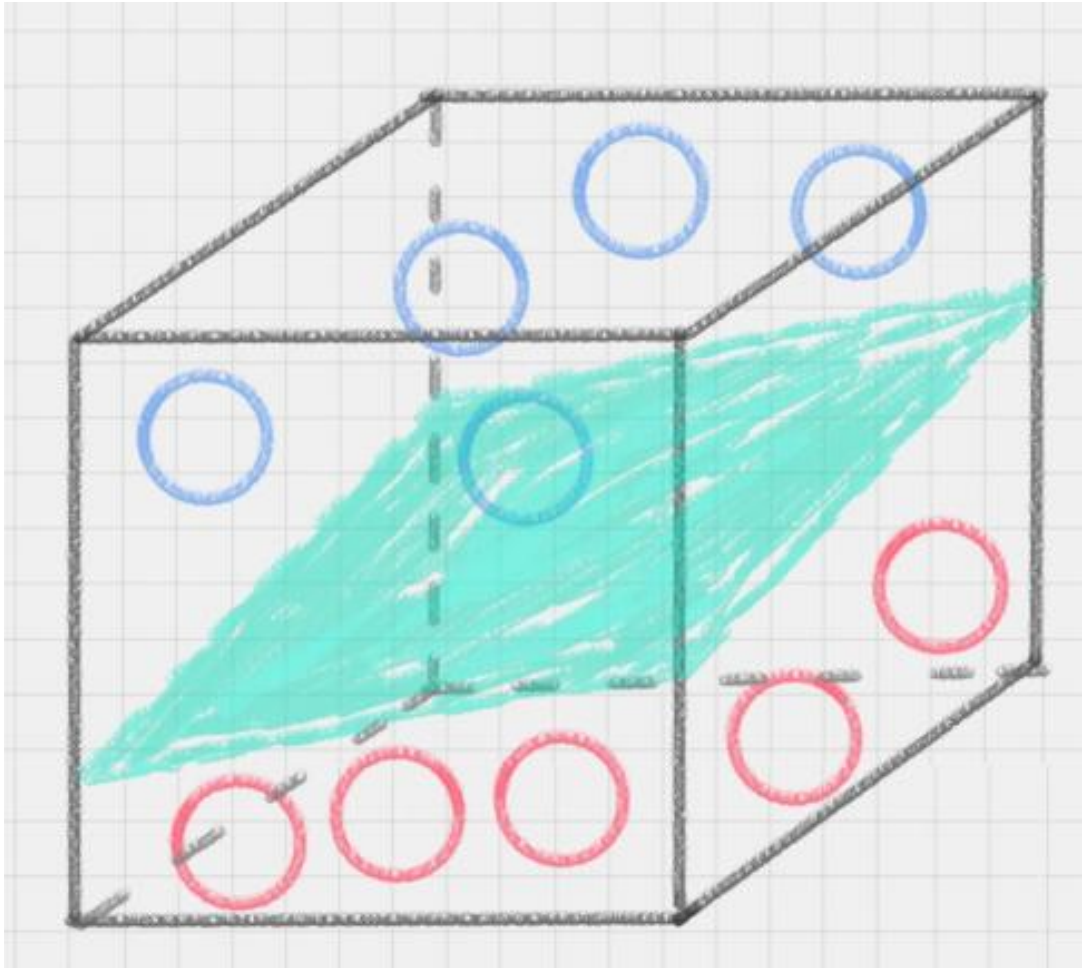
SVM支持向量机



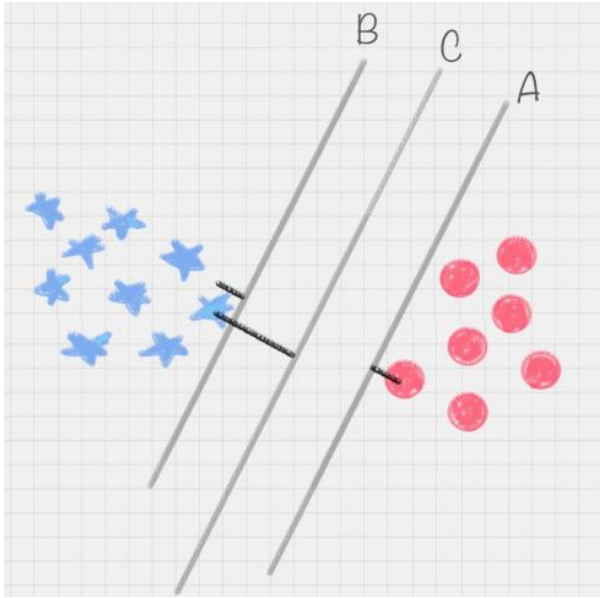




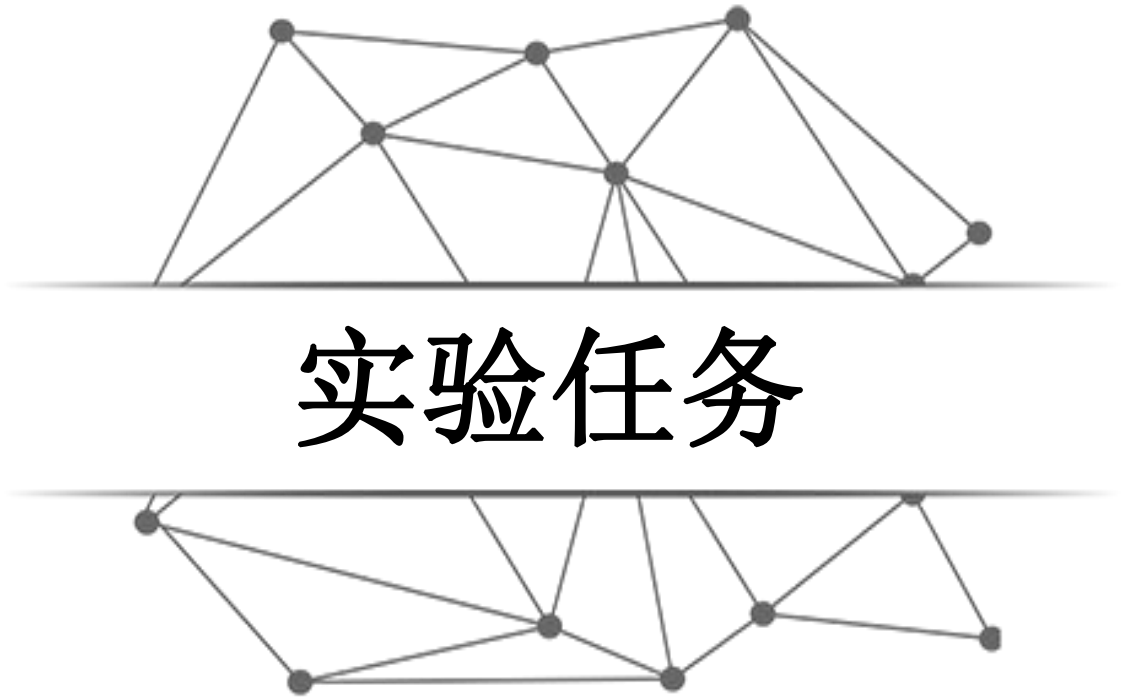
SVM支持向量机



支持向量机是一种**二分类**算法，通过在高维空间中构造超平面实现对样本的分类。



SVM 就是帮我们找到一个**超平面**，这个超平面能将不同的样本划分开，同时使得样本集中的点到这个分类超平面的**最小距离（即分类间隔）最大化。**



实验任务

实验任务

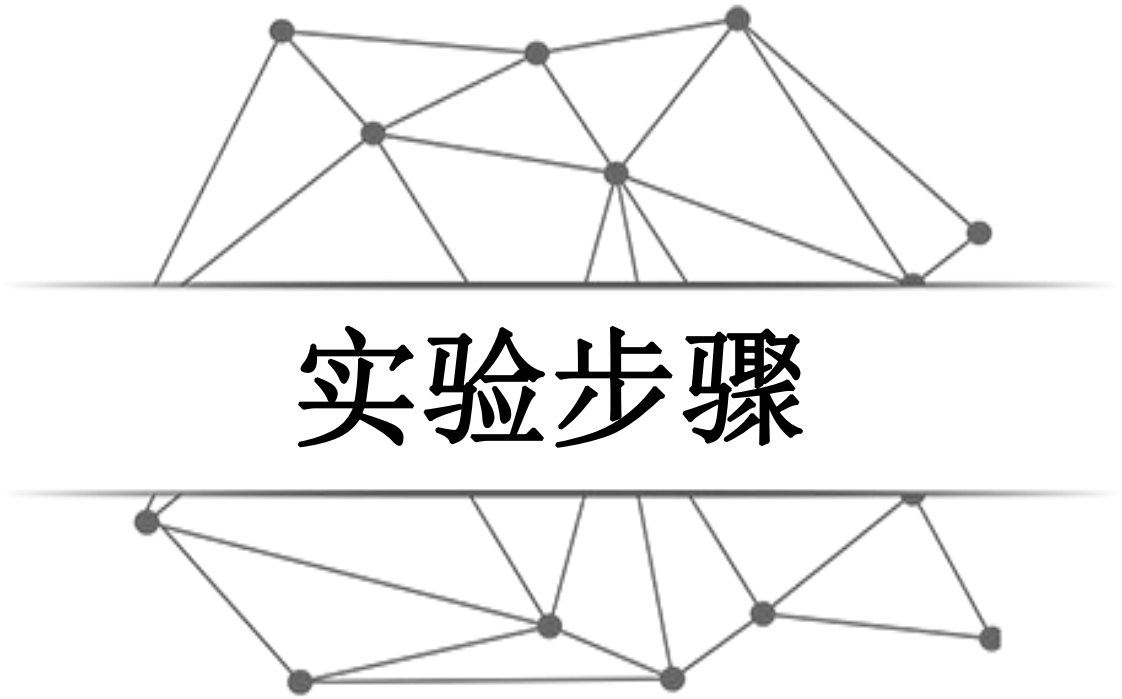

data.csv

用 SVM 进行乳腺癌检测,要求掌握分类任务的从头到尾的流程。

要求准确率达到baseline以上，即92.1058以上。

- mean 代表平均值，se 代表标准差，worst 代表最大值（3 个最大值的平均值）。
- 每张图像都计算了相应的特征，得出了这 30 个特征值（不包括 ID 字段和分类标识结果字段 diagnosis）
- 实际上是 10 个特征值（radius、texture、perimeter、area、smoothness、compactness、concavity、concave points、symmetry 和fractal_dimension_mean）的 3 个维度，平均、标准差和最大值。

字段	含义
ID	ID标识
diagnosis	M/B（M：恶性，B：良性）
radius_mean	半径（点中心到边缘的距离）平均值
texture_mean	文理（灰度值的标准差）平均值
perimeter_mean	周长 平均值
area_mean	面积 平均值
smoothness_mean	平滑程度（半径内的局部变化）平均值
compactness_mean	紧密度（=周长*周长/面积-1.0）平均值
concavity_mean	凹度（轮廓凹部的严重程度）平均值
concave points_mean	凹缝（轮廓的凹部分）平均值
symmetry_mean	对称性 平均值
fractal_dimension_mean	分形维数（=海岸线近似-1）平均值
radius_se	半径（点中心到边缘的距离）标准差
texture_se	文理（灰度值的标准差）标准差
perimeter_se	周长 标准差
area_se	面积 标准差
smoothness_se	平滑程度（半径内的局部变化）标准差
compactness_se	紧密度（=周长*周长/面积-1.0）标准差
concavity_se	凹度（轮廓凹部的严重程度）标准差
concave points_se	凹缝（轮廓的凹部分）标准差
symmetry_se	对称性标准差
fractal_dimension_se	分形维数（=海岸线近似-1）标准差
radius_worst	半径（点中心到边缘的距离）最大值
texture_worst	文理（灰度值的标准差）最大值
perimeter_worst	周长 最大值
area_worst	面积 最大值
smoothness_worst	平滑程度（半径内的局部变化）最大值
compactness_worst	紧密度（=周长*周长/面积-1.0）最大值
concavity_worst	凹度（轮廓凹部的严重程度）最大值
concave points_worst	凹缝（轮廓的凹部分）最大值
symmetry_worst	对称性 最大值
fractal_dimension_worst	分形维数（=海岸线近似-1）最大值



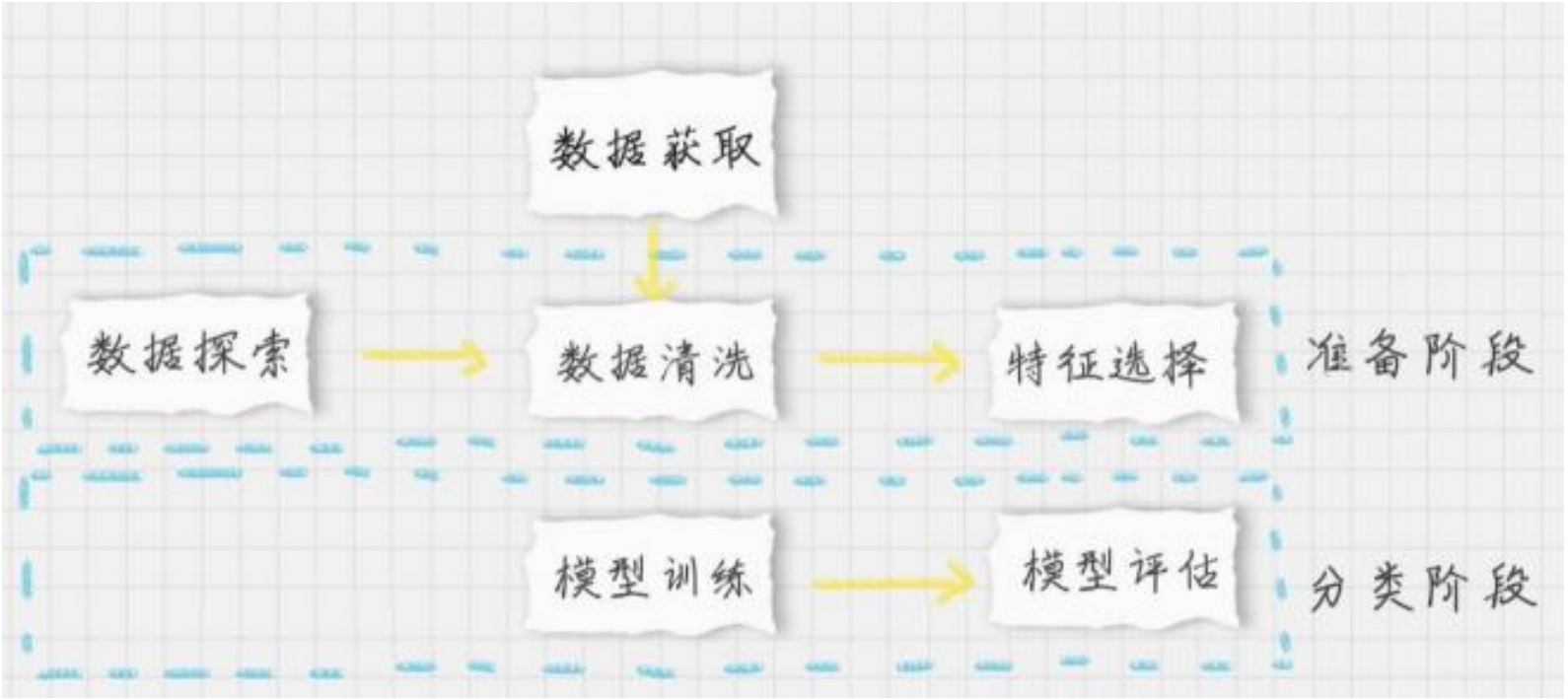
实验步骤

实验步骤

用 SVM 进行乳腺癌检测,要求掌握分类任务的从头到尾的流程。

要求准确率达到baseline以上，即92.1058以上。

1. 加载数据
2. 数据探索(数据可视化)
3. 特征选择
4. 使用SVM模型对训练集进行训练
5. 对验证集进行预测并计算准确率



加载数据

```
# 导入相关的包
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn import svm
from sklearn import metrics
from sklearn.preprocessing import StandardScaler
```

```
# 加载数据集，你需要把数据放到目录中
data = pd.read_csv("./data.csv")

# 数据探索
# 因为数据集中列比较多，我们需要把dataframe中的列全部显示出来
pd.set_option('display.max_columns', None)
print(data.columns)
#print(data.head(5))
print(data.describe())
```

```
Index(['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
       'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
       'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
       'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave points_worst',
       'symmetry_worst', 'fractal_dimension_worst'],
      dtype='object')
                 id  radius_mean  texture_mean  perimeter_mean     area_mean  \
count  5.690000e+02   569.000000    569.000000      569.000000    569.000000
mean   3.037183e+07    14.127292     19.289649       91.969033    654.889104
std    1.250206e+08     3.524049      4.301036       24.298981    351.914129
min    8.670000e+03     6.981000      9.710000       43.790000    143.500000
25%    8.692180e+05    11.700000     16.170000       75.170000    420.300000
50%    9.060240e+05    13.370000     18.840000       86.240000    551.100000
75%    8.813129e+06    15.780000     21.800000      104.100000    782.700000
max    9.113205e+08    28.110000     39.280000      188.500000   2501.000000

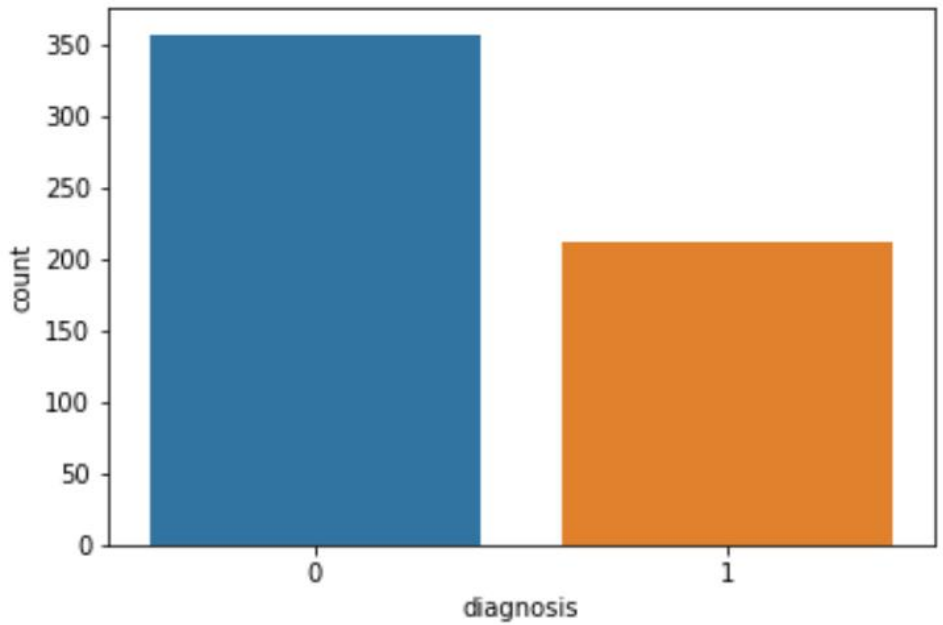
       smoothness_mean  compactness_mean  concavity_mean  concave points_mean
count       569.000000        569.000000      569.000000           569.000000
mean          0.096360          0.104341        0.088799             0.048919
std           0.014064          0.052813        0.079720             0.038803
min           0.052630          0.019380        0.000000             0.000000
25%           0.086370          0.064920        0.029560             0.020310
50%           0.095870          0.092630        0.061540             0.033500
75%           0.105300          0.130400        0.130700             0.074000
max           0.163400          0.345400        0.426800             0.201200
```

数据清洗

```
# 将特征字段分成3组
features_mean= list(data.columns[2:12])
features_se= list(data.columns[12:22])
features_worst=list(data.columns[22:32])
```

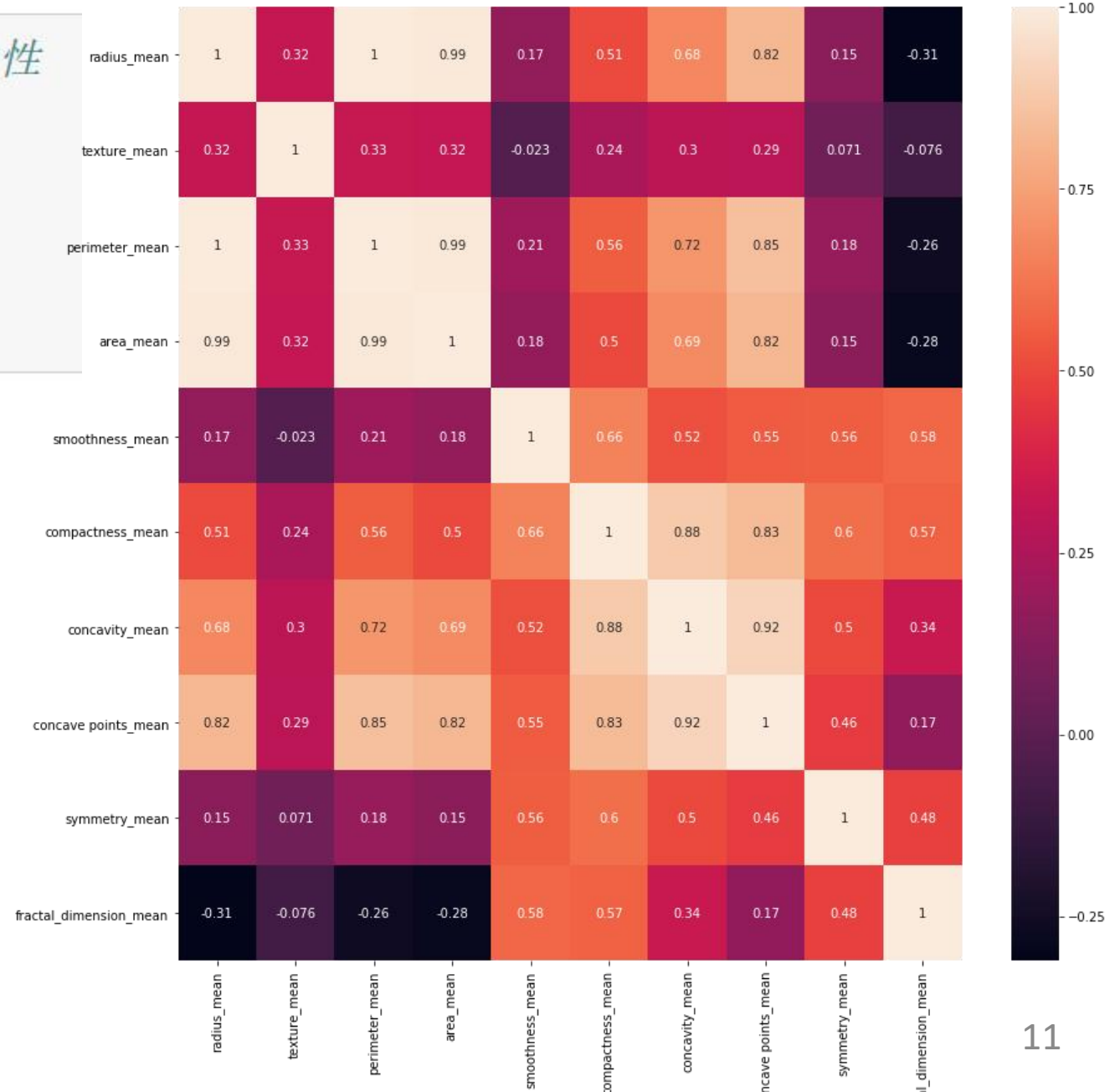
```
# 数据清洗
# ID列没有用，删除该列
data.drop("id",axis=1,inplace=True)
# 将B良性替换为0，M恶性替换为1
data['diagnosis']=data['diagnosis'].map({'M':1,'B':0})
```

```
# 将肿瘤诊断结果可视化
sns.countplot(data['diagnosis'],label="Count")
plt.show()
```



数据可视化

```
# 用热力图呈现features_mean字段之间的相关性
corr = data[features_mean].corr()
plt.figure(figsize=(14,14))
# annot=True显示每个方格的数据
sns.heatmap(corr, annot=True)
plt.show()
```



特征选择&数据划分

```
# 特征选择，这里可以选取其它特征
features_remain = ['radius_mean','texture_mean', 'smoothness_mean','compactness_mean','symmetry_mean', 'fractal_dimension_mean']
```

```
# 抽取30%的数据作为测试集，其余作为训练集
train, test = train_test_split(data, test_size = 0.2)
# 抽取特征选择的数值作为训练和测试数据
train_X = train[features_remain]
train_y=train['diagnosis']
test_X= test[features_remain]
test_y =test['diagnosis']
```

```
train_X.shape
```

```
(455, 6)
```

```
# 采用Z-Score规范化数据，保证每个特征维度的数据均值为0，方差为1
ss = StandardScaler()
train_X = ss.fit_transform(train_X)
test_X = ss.transform(test_X)
```

模型训练&模型预测

```
# 创建SVM分类器
model = svm.SVC(kernel='rbf',C=1,degree=3) # { 'linear', 'poly', 'rbf', 'sigmoid' }
# 用训练集做训练
model.fit(train_X,train_y)
# 用测试集做预测
prediction=model.predict(test_X)
print('准确率: ', metrics.accuracy_score(prediction,test_y))
```

```
准确率:  0.9210526315789473
```

- 中文文档：https://www.sklearncn.cn/5/#141
- 官方文档：https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html#sklearn.svm.SVC

- 参数 C 代表目标函数的惩罚系数，惩罚系数指的是分错样本时的惩罚程度，默认情况下为1.0。**当 C 越大的时候，分类器的准确性越高，但同样容错率会越低，泛化能力会变差。**相反，C 越小，泛化能力越强，但是准确性会降低。

SVM核函数

SVM 是通过核函数将样本从原始空间映射到一个更高维的特质空间中，这样就使得样本在新的空间中线性可分。

最常用的核函数有线性核、多项式核、高斯核、sigmoid核；

如何选择核函数：

当特征维数 d 超过样本数 m 时 (文本分类问题通常是这种情况), 使用线性核;

当特征维数 d 比较小. 样本数 m 中等时, 使用RBF核;

当特征维数 d 比较小. 样本数 m 特别大时, 支持向量机性能通常不如深度神经网络

名称	形式	优点	缺点
线性核	$\boldsymbol{x}_i^{\top}\boldsymbol{x}_j$	有高效实现, 不易过拟合	无法解决非线性可分问题
多项式核	$(\beta\boldsymbol{x}_i^{\top}\boldsymbol{x}_j+\theta)^n$	比线性核更一般, n 直接描述了被映射空间的复杂度	参数多, 当 n 很大时会导致计算不稳定
RBF 核	$\exp\left(-\frac{\|\boldsymbol{x}_i-\boldsymbol{x}_j\|^2}{2\sigma^2}\right)$	只有一个参数, 没有计算不稳定问题	计算慢, 过拟合风险大

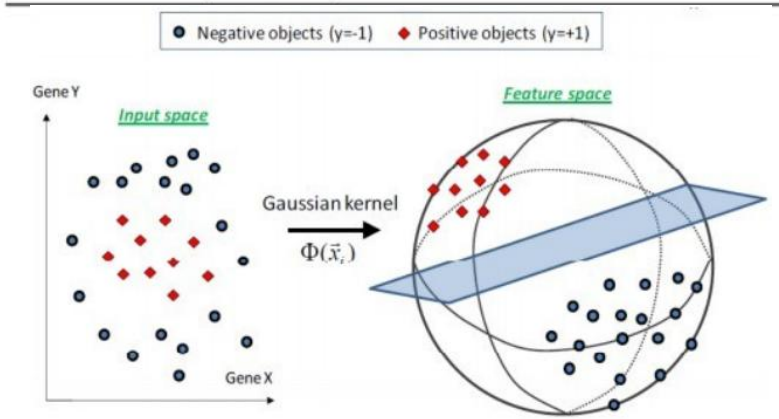
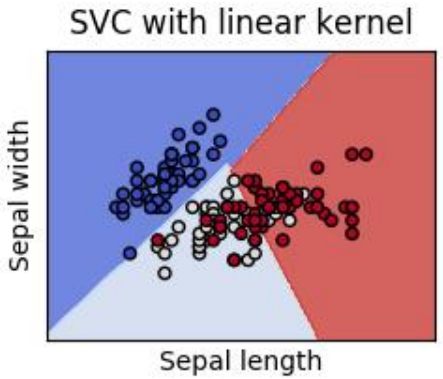
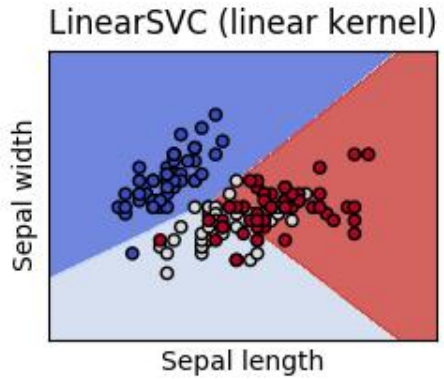
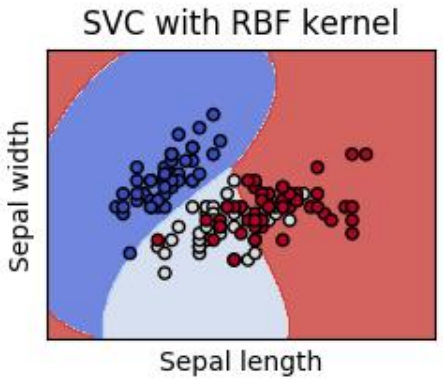
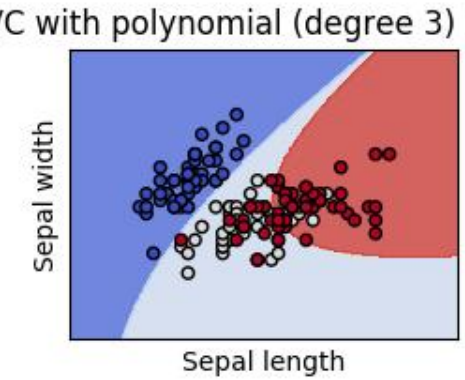


图 10: 高斯核示意









SVM总结

- 线性核函数，是在数据线性可分的情况下使用的，运算速度快，效果好。不足在于它不能处理线性不可分的数据。
- 多项式核函数可以将数据从低维空间映射到高维空间，但参数比较多，计算量大。
- 高斯核函数同样可以将样本映射到高维空间，但相比于多项式核函数来说所需的参数比较少，通常性能不错，所以是默认使用的核函数。
- 了解深度学习的同学应该知道 sigmoid 经常用在神经网络的映射中。因此当选用 sigmoid核函数时，SVM 实现的是多层神经网络。

关于 SVM 分类器的三个程度：

1. 完全线性可分情况下的线性分类器，也就是线性可分的情况，是最原始的 SVM，它最核心的思想就是找到最大的分类间隔；

2. 大部分线性可分情况下的线性分类器，引入了软间隔的概念。软间隔，就是允许一定量的样本分类错误；

3. 线性不可分情况下的非线性分类器，引入了核函数。它让原有的样本空间通过核函数投射到了一个高维的空间中，从而变得线性可分。

THANK YOU FOR WATCHING

梁威鹏